C Quick Reference and Debugging Examples

Basic types and output

```
#include <stdio.h>
int main(void) {
    int count = 3;
    printf("%d\n", count);
    return 0;
}
```

Use the matching printf format specifier. %d prints an int; %f prints a floating-point value.

Array bounds

```
int values[] = {10, 20, 30};
int length = 3;
for (int i = 0; i < length; i++) {
    printf("%d\n", values[i]);
}
```

C arrays are zero-indexed. The valid indexes here are 0, 1, and 2. A condition i <= length accesses beyond the array and causes undefined behavior.

Pointers

```
int value = 7;
int *ptr = &value;
if (ptr != NULL) {
    printf("%d\n", *ptr);
}
```

A pointer must refer to a valid object before dereferencing. Dereferencing NULL or an uninitialized pointer has undefined behavior.

Strings

```
#include <string.h>
char text[16] = "hello";
size_t n = strlen(text);
```

Strings in C are character arrays terminated by ’\0’. Reserve space for the terminator and avoid writing beyond the array.

Dynamic allocation

```
#include <stdlib.h>
int *items = malloc(3 * sizeof *items);
if (items != NULL) {
    items[0] = 42;
    free(items);
}
```

Check allocation results and release allocated memory with free exactly once when it is no longer needed.